

製品検査システム開発仕様書

1. システム概要

本システムは、自社開発の生産管理システム（PM）の機能を拡張し、AI画像認識（YOLOv8）と電子天秤を活用した製品検査パッケージである。キーエンス製カメラセンサーの高コスト構成を、Jetson Orin Nano + Python + Dockerによる内製構成へリプレイスし、既存のDjango/Vue.js/MySQL環境とシームレスに連携させる。

対象ユースケース

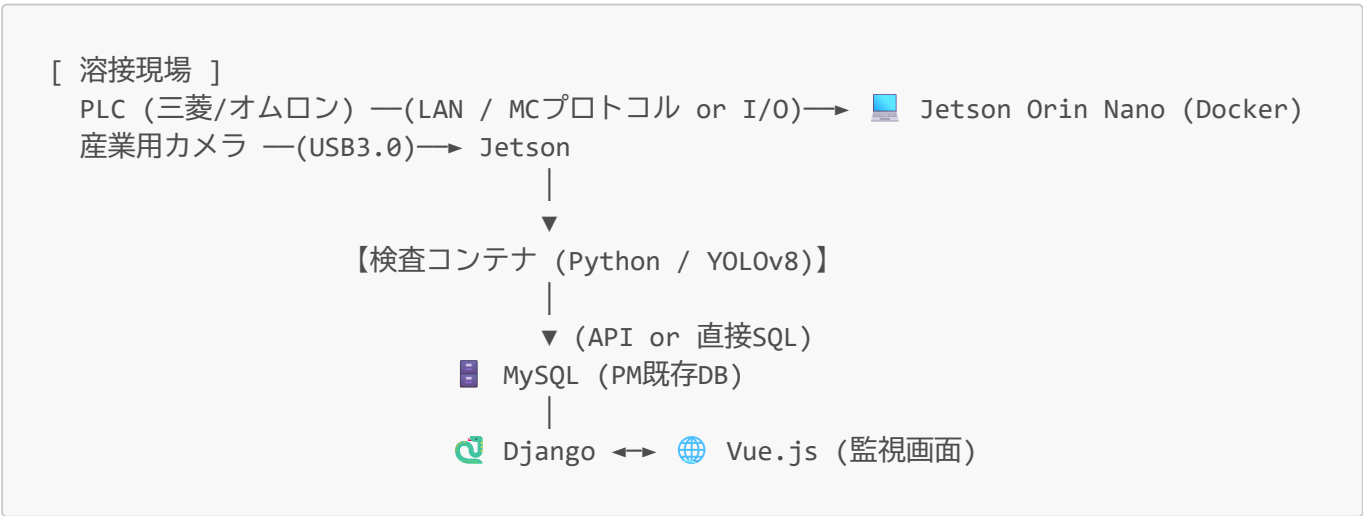
#	シーン	検査内容	デバイス
1	溶接ライン（誤品防止）	ナットサイズ判定（M6/M8/M10等）	産業用カメラ + Jetson + PLC連携
2	出荷準備（最終検品）	品種判定 + 数量カウント	スマホカメラ + 電子天秤

2. ユースケース①：溶接ライン誤品防止

2.1 業務フロー

1. 作業者が母材にナットをセットする
2. 溶接ロボット起動準備（PLCから「検査開始」信号）
3. カメラがナットを撮影 → YOLOv8がサイズを瞬時に判定
4. 判定結果をPLCへ返す
 - **OK**: ロボット起動許可
 - **NG**: インターロック（起動禁止） + パトライト点灯 + ブザー
5. 判定結果・NG画像をMySQLへ自動保存

2.2 システム構成



2.3 ハードウェア構成・概算コスト

機器	仕様	概算費用
Jetson Orin Nano 開発者キット	8GB RAM, AI処理専用	約90,000円
産業用USB3.0カメラ	Basler等	約50,000円
カメラレンズ・照明	バックライト or リング照明推奨	約30,000円
周辺機器（電源、SSD等）	-	約15,000円
合計（1ラインあたり）		約185,000円

※ キーエンス構成（50万～200万円以上）と比較して1/3～1/10のコスト

2.4 PLC通信方式

既存PLCは三菱電機（MELSEC）またはオムロン。

方式A: USB型デジタルI/Oモジュール（難易度：低）

- CONTEC社製等のUSB I/Oモジュールを使用
- PLCの24V信号を直接結線
- PLC側のラダー変更は最小限（出力ON/OFF数行の追加のみ）

```
# PLC信号の監視と結果出力（イメージ）
import cdio

dio = cdio.Dio()
dio.open("DI0000")

while True:
    if dio.input_bit(1) == 1: # 検査開始トリガー
        result = run_yolo_inspection() # YOLO判定実行
        if result == "OK":
            dio.output_bit(2, 1) # OK信号
        else:
            dio.output_bit(3, 1) # NG信号
        break
```

方式B: MCプロトコル/LAN通信（難易度：中）

- 三菱MELSEC: `pymelsec` ライブラリでDレジスタ読み書き
- オムロン: `pycomm3` でEtherNet/IP通信
- LANケーブル1本で完結、文字列データも送受信可能

```
# 三菱PLCとのLAN通信（イメージ）
from pymelsec import TypeQ3E

with TypeQ3E(host='192.168.0.10', port=5007) as plc:
    trigger = plc.read('D100') # 検査トリガー監視
```

```
if trigger == 1:
    result_ok = run_yolo_inspection()
    plc.write('D101', 1 if result_ok else 0)
```

2.5 現場設置の注意事項

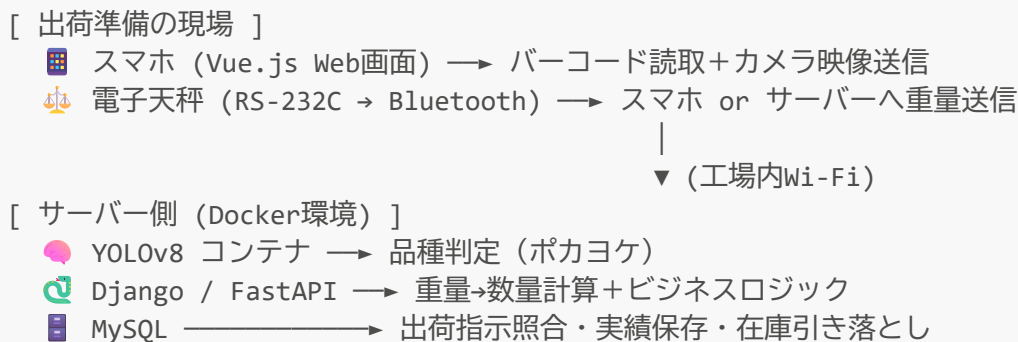
- **スパッタ対策**: カメラを防塵ハウジングに収納＋交換可能な保護ガラス＋エアブロー
- **溶接アーク光対策**: 遮光カーテンの影にカメラ設置、溶接中はシャッター閉
- **照明**: 金属反射を抑える同軸落射照明 or ドーム照明を推奨
- **バックライト**: ナットをシルエット化すれば外径・内径の差が明確になり最高精度

3. ユースケース②：出荷準備（最終検品）

3.1 業務フロー

1. 作業者がスマホで出荷指示バーコードを読み取る
2. Vue.js画面に「製品A：50個、単重15.5g」等の情報がセットされる
3. **品種判定**: スマホカメラで製品にかざす → YOLOv8が品種を識別
 - 異品混入を検知した場合は画面に警告、計量プロセスへ進めないようロック
4. **数量カウント**: 製品を電子天秤に載せる → 重量から個数を自動計算
 - 計算式: $\text{総重量(g)} \div \text{単重(g)} = \text{個数}$
5. 品種OK＋数量一致で「出荷確定」ボタンが活性化
6. 確定時にMySQLの在庫引き落とし＋エビデンス画像保存

3.2 システム構成



3.3 品種判定の実装方式

スマホのブラウザ（Vue.js）からカメラを起動し、サーバー側でAI処理を行う。

```
// Vue.js: スマホの背面カメラを起動
const constraints = {
  video: { facingMode: { exact: "environment" } }
};
navigator.mediaDevices.getUserMedia(constraints)
  .then((stream) => {
```

```
video.srcObject = stream;
// 定期的にフレームをキャプチャしてサーバーへ送信
});
```

3.4 数量カウント（重量計数）の実装

電子天秤の接続方式

方式	仕組み	メリット
Bluetooth化	RS-232C → Bluetoothアダプタ	配線不要、スマホと直結（Web Bluetooth API）
Wi-Fi/LAN化	シリアル→Wi-Fi変換器	サーバーへ直接送信、安定性重視

Django側の数量判定ロジック

```
def check_quantity(product_code, total_weight, expected_qty):
    product = ProductMaster.objects.get(product_code=product_code)
    calculated_qty = round(total_weight / product.unit_weight)

    if calculated_qty == expected_qty:
        return {"status": "OK", "quantity": calculated_qty}
    else:
        diff = expected_qty - calculated_qty
        direction = "不足" if diff > 0 else "過剰"
        return {
            "status": "NG",
            "message": f"{direction}しています。差分: {abs(diff)}個"
        }
```

3.5 エビデンス（証拠）画像の保存

- NG画像および出荷確定時の画像をサーバー（Djangoのメディアフォルダ）に保存
- MySQLにはファイルパス（文字列）のみを保存（DBの肥大化防止）
- ロット番号・日時・担当者名と紐づけてトレーサビリティを確保

4. データベース設計（MySQL拡張）

4.1 製品マスターへの項目追加

```
ALTER TABLE product_master
    ADD COLUMN unit_weight DECIMAL(8, 2) COMMENT '1個あたりの基準重量(g)',
    ADD COLUMN weight_tolerance DECIMAL(4, 2) DEFAULT 0.05 COMMENT '重量の許容誤差率
(例: 0.05 = ±5%)';
```

4.2 検査ログテーブル（溶接ライン用）

```
CREATE TABLE inspection_logs (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  work_id VARCHAR(50) COMMENT '母材シリアル/ロット番号',  
  product_type VARCHAR(20) COMMENT '生産型番',  
  expected_spec VARCHAR(10) COMMENT '本来のサイズ (例: M8) ',  
  detected_spec VARCHAR(10) COMMENT '検知したサイズ (例: M6) ',  
  judgment VARCHAR(10) NOT NULL COMMENT 'OK / NG',  
  ng_image_path VARCHAR(255) COMMENT 'NG画像の保存パス',  
  inspection_time_ms INT COMMENT '判定にかかった時間(ms)',  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

4.3 出荷検査ログテーブル（出荷準備用）

```
CREATE TABLE shipping_inspection_logs (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  shipping_instruction_id VARCHAR(50) NOT NULL COMMENT '出荷指示ID',  
  product_code VARCHAR(50) NOT NULL COMMENT '製品型番',  
  expected_qty INT NOT NULL COMMENT '指示数量',  
  measured_weight DECIMAL(10, 2) NOT NULL COMMENT '天秤の実測総重量(g)',  
  calculated_qty INT NOT NULL COMMENT '重量から計算した個数',  
  judgment VARCHAR(10) NOT NULL COMMENT 'OK / NG',  
  evidence_image_path VARCHAR(255) COMMENT 'エビデンス画像の保存パス',  
  operator_name VARCHAR(50) COMMENT '作業担当者',  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

5. AI（YOLOv8）モデル開発

5.1 ナット検査モデル

- **対象クラス:** M6, M8, M10 等（サイズごとに1クラス）
- **学習画像:** 母材にナットをセットした状態の写真を各サイズ数百枚
- **データオーグメンテーション:** 角度変化、明るさ変化、ぼかし等を適用
- **推論速度目標:** Jetson Orin Nanoで30FPS以上

5.2 出荷製品判定モデル

- **対象クラス:** 出荷対象の製品型番ごとに1クラス
- **学習画像:** スマホで撮影した製品画像（様々な角度・明るさ）
- **手ブレ対策:** 学習データにぼけ・傾き画像を意図的に混入

5.3 モデル開発手順

1. **画像収集:** 現場でスマホ・Webカメラで撮影（各クラス数百枚〜）
2. **アノテーション:** バウンディングボックスのラベル付け

3. **学習**: PCでYOLOv8をファインチューニング → **best.pt** を出力
4. **検証**: テスト画像で精度・速度を確認
5. **デプロイ**: Dockerコンテナ内にモデルを配置

6. ハードウェア選定ガイド

6.1 カメラインターフェース

方式	距離	電源	用途
USB 3.0 (USB3 Vision)	5m以内	バスパワー（不要）	推奨。ケーブル1本で最もシンプル
GigE (有線LAN)	100mまで	PoE給電ハブ	カメラとPCが離れている場合
Wi-Fi	-	別途必要	非推奨 （遅延・フレーム落ち・工場電波ノイズ）

6.2 PC/エッジデバイス

用途	推奨構成
OpenCVのみ（AIなし）	Core i5 / 8GB RAM / SSD 256GB / GPU不要
YOLOv8リアルタイム推論	Core i7 / 16GB RAM / NVIDIA RTX 4060以上
現場設置（省スペース）	Jetson Orin Nano （約9万円、名刺～手のひらサイズ）
現場設置（高性能）	Jetson Orin NX（約15～22万円）

6.3 産業用カメラメーカー

- **Basler**（バスラー）：世界シェアトップ。Python公式SDK **pypylon** あり
- **IDS**（アイディーエス）
- **オムロン（Sentech）** / **東芝テリー**

6.4 電子天秤

- A&D社製等（RS-232Cポート付き）
- Bluetooth変換アダプタ（数千円～1万円）でワイヤレス化

7. コスト比較

項目	キーエンス（専用センサー）	自作（Python/Docker）
コントローラー/PC	専用コントローラー（高額）	Jetson Orin Nano 約9万円
カメラ	専用カメラ（数万～数十万円）	産業用USB3.0カメラ 約5万円
ソフトウェア	専用ソフト（ライセンス費用）	Python + OpenCV + YOLOv8（無料）

項目	キーエンス（専用センサー）	自作（Python/Docker）
実績保存	CSV/上位リンク（面倒）	MySQL直接保存、Vue.jsで即可視化
カスタマイズ	メーカー依頼（追加費用）	自社でPythonコード修正（無料）
1ラインあたり合計	50万～200万円以上	約18.5万円

8. 開発ロードマップ

Phase 1: プロトタイプ検証（出荷準備）

- ☐ 製品マスターに `unit_weight` 項目を追加
- ☐ スマホカメラ起動のVue.js画面を作成（getUserMedia）
- ☐ YOLOv8で製品判定のPoC（手持ちのPC + Webカメラ）
- ☐ 電子天秤のBluetooth接続テスト

Phase 2: 出荷検品システム構築

- ☐ Django API: 重量→数量計算エンドポイント
- ☐ Vue.js: 出荷検品専用画面（バーコード読取→品種判定→計量→確定）
- ☐ MySQL: `shipping_inspection_logs` テーブル作成
- ☐ エビデンス画像の保存・表示機能

Phase 3: 溶接ライン検査システム構築

- ☐ Jetson Orin Nano 調達・Docker環境構築
- ☐ 産業用カメラ+照明の選定・設置
- ☐ ナットサイズ判定用YOLOv8モデルの学習
- ☐ PLC通信（I/O or MCプロトコル）の実装
- ☐ MySQL: `inspection_logs` テーブル作成

Phase 4: Dockerパッケージ化・横展開

- ☐ 全コンテナのdocker-compose構成整備
- ☐ 他ラインへの横展開
- ☐ 外販パッケージとしての整備（将来）